

Практическая работа. Модуль 4 (экзамен)

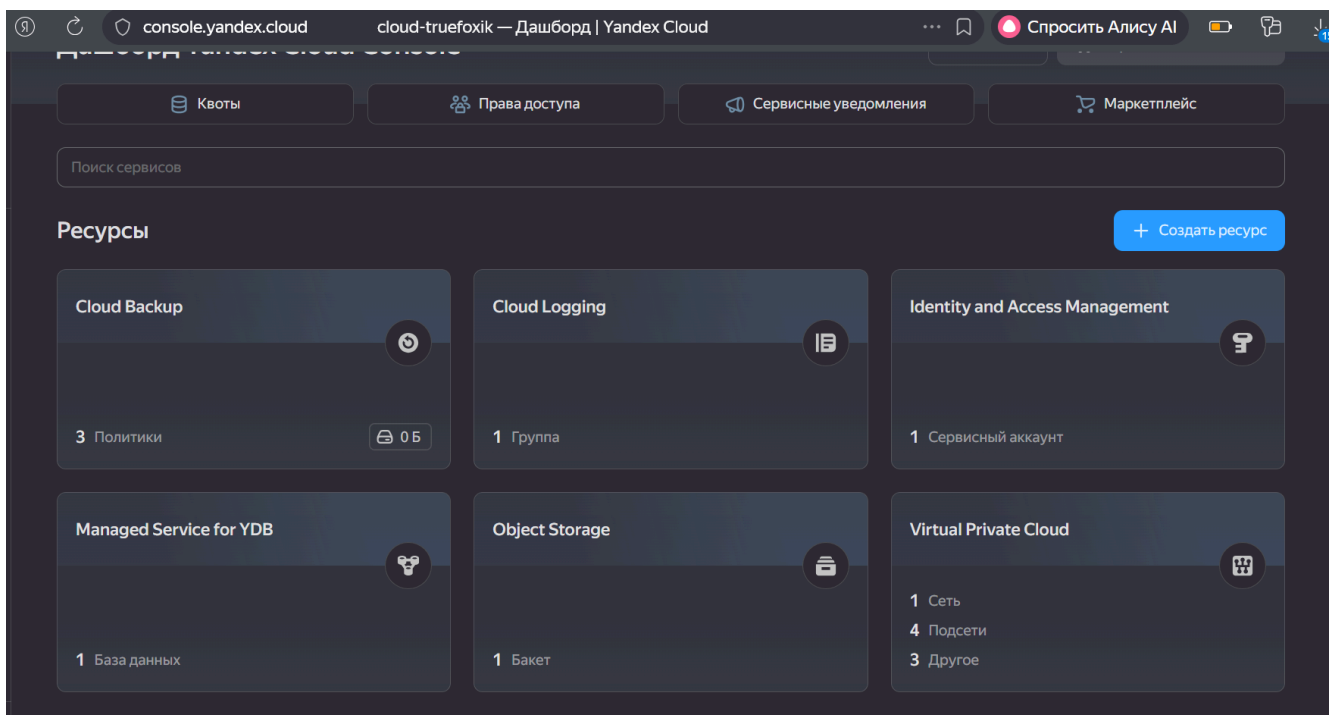
Выполнила Макаркина Татьяна Сергеевна

Задание 1. Работа с Yandex Data Transfer

Перед началом работы создам сервисный аккаунт. Пока выдам ему только роли, необходимые для данного задания. Это точно `ydb.admin` - для работы с базой данных, `storage.editor` - для загрузки данных в бакет и `ycs.user` - для работы внутри моей сети. Далее, соответственно, создаю сеть и бакет.

Следующий шаг - создаю базу данных YDB.

Итого на данном этапе все мои ресурсы:



Теперь необходимо подключиться к YDB. WebSQL у этого ресурса нет, поэтому буду использовать PowerShell. YandexCLI у меня уже установлен локально. Дополнительно нужно установить именно YDB.

Один из вариантов для подключения к YDB - использование токена. Поэтому создам его и сохраню локально


```
truef@Foxik:/mnt/d/Dz$ ydb \
--endpoint grpc://ydb.serverless.yandexcloud.net:2135 \
--database /ru-central1/blgon3vimj806plgnkba/etnrule9567ot4k46ans \
--iam-token-file iam-token.txt \
scheme ls -l
```

Type	Owner	Size	Created	Modified	Name
dir	metadata@system				.sys

```
truef@Foxik:/mnt/d/Dz$ ydb --endpoint grpc://ydb.serverless.yandexcloud.net:2135 --database /ru-central1/blgon3vimj806plgnkba/etnrule9567ot4k46ans --iam-token-file iam-token.txt yql -f create-tables.yql
truef@Foxik:/mnt/d/Dz$ ydb --endpoint grpc://ydb.serverless.yandexcloud.net:2135 --database /ru-central1/blgon3vimj806plgnkba/etnrule9567ot4k46ans --iam-token-file iam-token.txt scheme ls -l
```

Type	Owner	Size	Created	Modified	Name
dir	metadata@system				.sys
table	ajee7bvvmfhu8k8h96@as	0B	Fri, 12 Jun 2026 14:12:23 MSK	Unknown	mental_health

Вот сам код скрипта:

```
> truef@Foxik: /mnt/d/Dz × + v
GNU nano 7.2 create-tables.yql
CREATE TABLE IF NOT EXISTS mental_health (
  participant_id Utf8,
  age Uint32,
  gender Utf8,
  country Utf8,
  occupation Utf8,
  work_hours_per_week Uint32,
  screen_time_hours Float,
  sleep_hours Float,
  sleep_quality Utf8,
  exercise_frequency Utf8,
  stress_score Uint32,
  anxiety_score Uint32,
  depression_score Uint32,
  social_support Utf8,
  therapy_history Utf8,
  family_history_mental_illness Utf8,
  academic_or_job_pressure Uint32,
  financial_stress_score Uint32,
  mental_health_risk Utf8,
  survey_date Timestamp,
  PRIMARY KEY(participant_id));
```

Следующий шаг - заполнить таблицу данными. В YDB есть возможность просто импортировать данные из csv-файла - так и сделаю

```
truef@Foxik:/mnt/d/Dz$ touch insert-values.yql
truef@Foxik:/mnt/d/Dz$ nano insert-values.yql
truef@Foxik:/mnt/d/Dz$ ydb \
--endpoint grpc://ydb.serverless.yandexcloud.net:2135 \
--database /ru-central1/blgon3vimj806plgnkba/etnrule9567ot4k46ans \
--iam-token-file iam-token.txt \
import file csv \
-p mental_health \
--header \
mental_health_survey_dataset.csv
100% | completed | 1.44 MiB/s | 39 MiB / 39 MiB
Elapsed: 27s. Total read size: 39 MiB. Average speed: 1.44 MiB/s.
```

Проверю схему данных: размер таблицы изменился

```
truef@Foxik:/mnt/d/Dz$ ydb --endpoint grpc://ydb.serverless.yandexcloud.net:2135 --database /ru-central1/b1gon3vimj806p1gnkba/etnrule9567ot4k46ans --iam-token-file iam-token.txt scheme ls -l
```

Type	Owner	Size	Created	Modified	Name
dir	metadata@system				.sys
table	ajee7bvvmfhu8k8h96@as	74.6MB	Fri, 12 Jun 2026 14:12:23 MSK	Fri, 12 Jun 2026 14:23:48 MSK	mental_health

```
truef@Foxik:/mnt/d/Dz$ |
```

Данные по колонкам заполнились корректно:

```
truef@Foxik:/mnt/d/Dz$ ydb \
--endpoint grpc://ydb.serverless.yandexcloud.net:2135 \
--database /ru-central1/b1gon3vimj806p1gnkba/etnrule9567ot4k46ans \
--iam-token-file iam-token.txt \
yql -s "SELECT * FROM mental_health LIMIT 5"
```

aca dem ic_ or_ job_ pr ess ure	age	anx iet y_ s cor e	cou ntr y	depr essi on_ s cor e	exer cise _fre quen cy	fam ily _hi sto ry_ men tal _il lne ss	finan cial_ stres s_sco re	gend er	ment al_h ealt h_ri sk	occ upa tio n	part icip ant_ id	scre en_t ime_ hour s	sleep _hour s	sle ep_ qua lit y	soc ial _su ppo rt	stre ss_ s cor e	sur vey _da te	the rap y_ h ist ory	work _hou rs_ p er_ w eek
10	36	1	"S audi Ar abia"	8	"Nev er"	"Ye s"	1	"Pre fer not to say"	"Hig h"	"Fr eel anc er"	"000 0217 e-bb 2e-4 309- 9db3 -b27 87b1 7a6d 5"	13.6	3.5	"Av era ge"	"St ron g"	10	"20 26- 02- 18T 00: 00: 00. 000 000 Z"	"No "	68

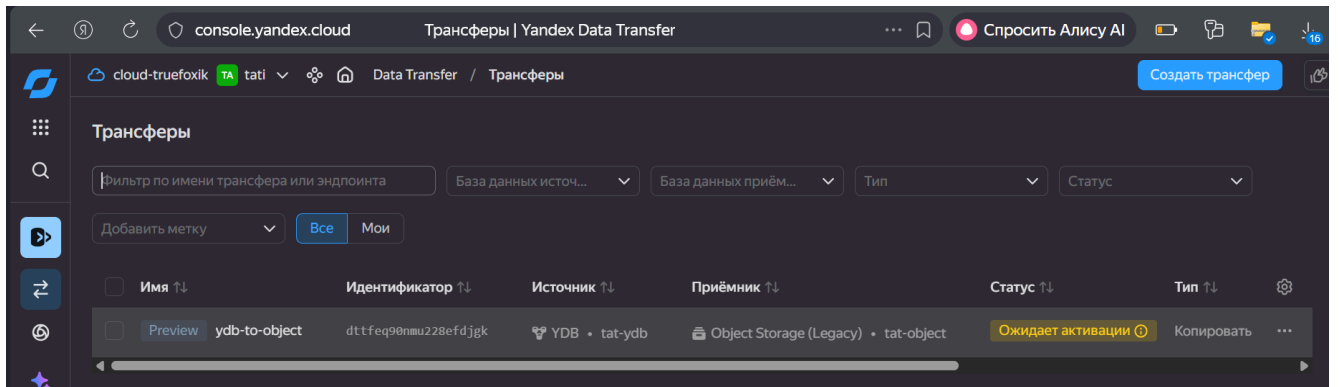
Последний шаг - создать, настроить и запустить трансфер данных из YDB в бакет Object Storage.

Эндпоинты созданы:

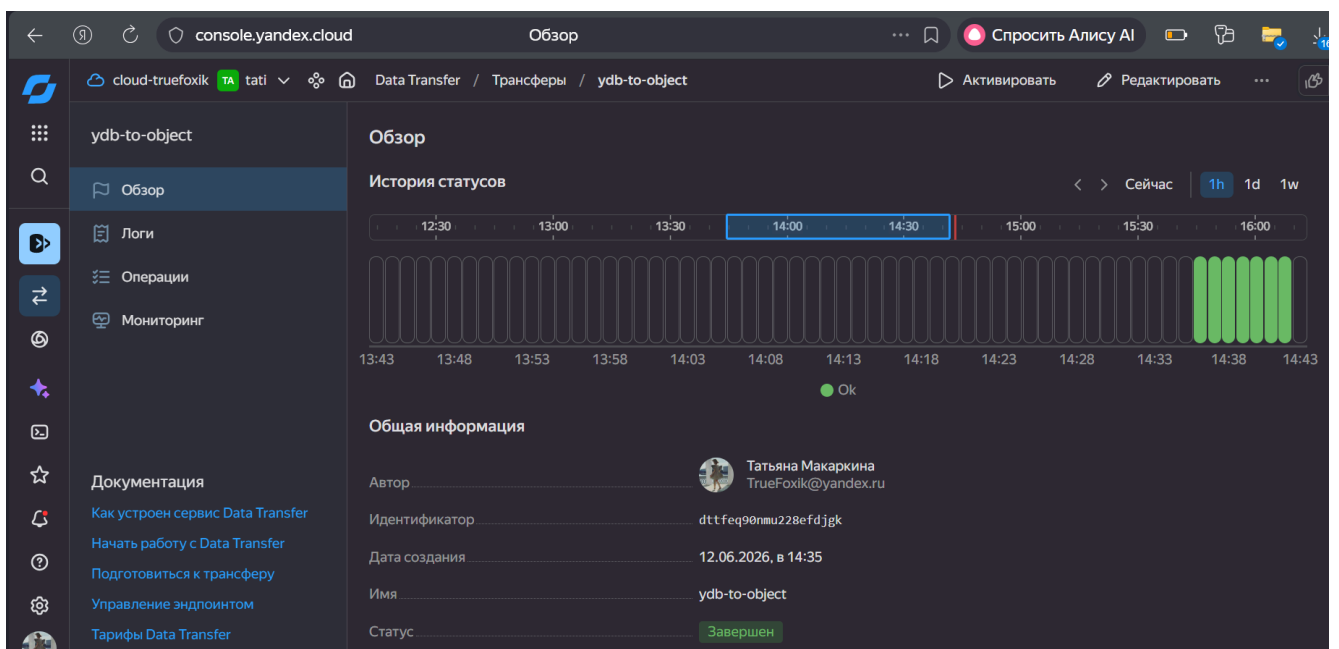
The screenshot shows the 'Endpoints' section of the Yandex Data Transfer console. Two endpoints are listed:

Имя	Идентификатор	Статус	Тип	База данных	Описание	Дата создания	Автор	Метки
tat-object	dtemvp9rd6utjqg5k871	Создан	Приёмник	Object Storage (Legacy)	—	12.06.2026, в 14:34	TrueFoxik@yandex.ru	—
tat-ydb	dtesbiqmcke5lk8vdgs0	Создан	Источник	YDB	—	12.06.2026, в 14:32	TrueFoxik@yandex.ru	—

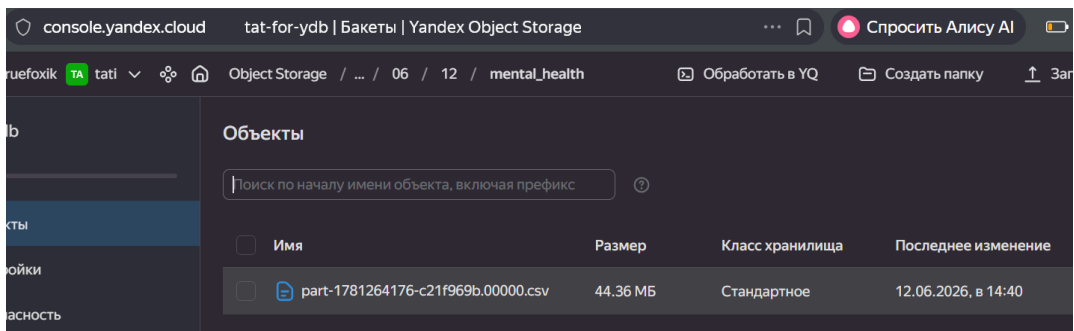
И создан трансфер. Осталось его запустить



Трансфер успешно завершен



Вот мои данные в бакете:



Итог: данные успешно перенесены из Managed Service for YDB в объектное хранилище Object Storage

Задание 2. Автоматизация работы с Yandex Data Processing при помощи Apache Airflow

Данные для данного задания я также беру с Kaggle (NYC Taxi Trip Duration <https://www.kaggle.com/datasets/yasserh/nyc-taxi-trip-duration>). Объем файла соответствует заданию.

Имя	Дата изменения	Тип	Размер
Вчера			
 NYC.csv	12.06.2026 16:05	Исходный файл Co...	195 888 КБ

Предварительно также создала бакет Object Storage. Внутри бакета у меня две папки - in и out. В первой у меня будут входные данные (загрузила туда csv). Во второй папке будут уже данные, обработанные с помощью кластера Dataproc.

Далее опять создаю сервисный аккаунт с нужными правами для работы с бакетом, сетью и кластером Dataproc. И саму сеть с подсетями и NAT-шлюзом

Следующий шаг - я создала кластеры Apache Hive Metastore и Managed Service for Apache Airflow с базовыми настройками (согласно документации Yandex Cloud). Кластеры запустились и живы



cloud-truefoxik  tati 

Yandex MetaData Hub / Metastore-сервер / Кластеры / metastore291  Подключиться  Редактировать 

metastore291 

 Обзор

 Обслуживание

 Логи

 Мониторинг

 Операции

 Права доступа

 Документация

 О кластерах Metastore

 Экспорт и импорт метаданных

 Подключение Yandex Data Processing к Metastore

Обзор

Имяmetastore291

Идентификаторc9q53qode1saanmhhdmn

Дата создания13.06.2026, в 12:16

Версия3.1

IP-адрес10.128.0.15

Сервисный аккаунттat

Сеть

Облачная сетьmodule-tat

Подсетиmodule-tat-ru-central1-a

Зона доступностиru-central1-a

Группы безопасностиdefault-sg-enpvbfrurpthr3emhuq

Доступность 

Все хосты работают нормально, все запущенные операции были успешно выполнены.

5 082,62 Р в месяц

Тарифы и цены

 С 1 мая 2026 года изменилась стоимость ряда сервисов. [Подробнее в блоге](#)

MetaData Hub. Вычислительные ресурсы кластера Apache Hive™ Metastore, 100% vCPU 3 312,00 Р

MetaData Hub. Вычислительные ресурсы кластера Apache Hive™ Metastore, RAM 1 770,62 Р

The screenshot shows the 'Overview' page for the 'airflow819' cluster. The cluster is 'Alive'. Key details include: Name: airflow819, Identifier: c9qvvv39v1hnjjd7q2m1, Creation Date: 13.06.2026, Airflow Version: 2.10, Python Version: 3.12, Service Account: tat, and Web Server: [Перейти](#). The status is 'Доступность: Alive' with a note: 'Все хосты работают нормально, все запущенные операции были успешно выполнены.' On the right, pricing is shown for vCPU (9 725,18 ₺ — 29 175,55 ₺) and RAM (4 147,20 ₺ — 12 441,60 ₺). A note mentions a price change starting May 1, 2026.

В бакете создала папки `scripts` и `dags`. Туда нужно будет положить файл с DAG и файл с PySpark-заданием. DAG я подготовила на основании документации, добавив свой SSH, идентификаторы подсети, сервисного аккаунта и кластера Nive.

Мое PySpark задание предполагает загрузку данных в csv-формате из бакета (папка `in`), их обработку и выгрузку в формате `parquet` (т.к. объем файлов достаточно большой) в тот же бакет (папка `out`).

Первая часть задания - это импорт необходимых мне инструментов, создание сессии, пути ввода/вывода и описание схемы данных.

```
truf@Foxik: /mnt/d/Dz
GNU nano 7.2 pyspark-dataproc.py *
from pyspark.sql import SparkSession
from pyspark.sql.types import *
from pyspark.sql.functions import col, when, lit, unix_timestamp, datediff, abs as pyspark_abs

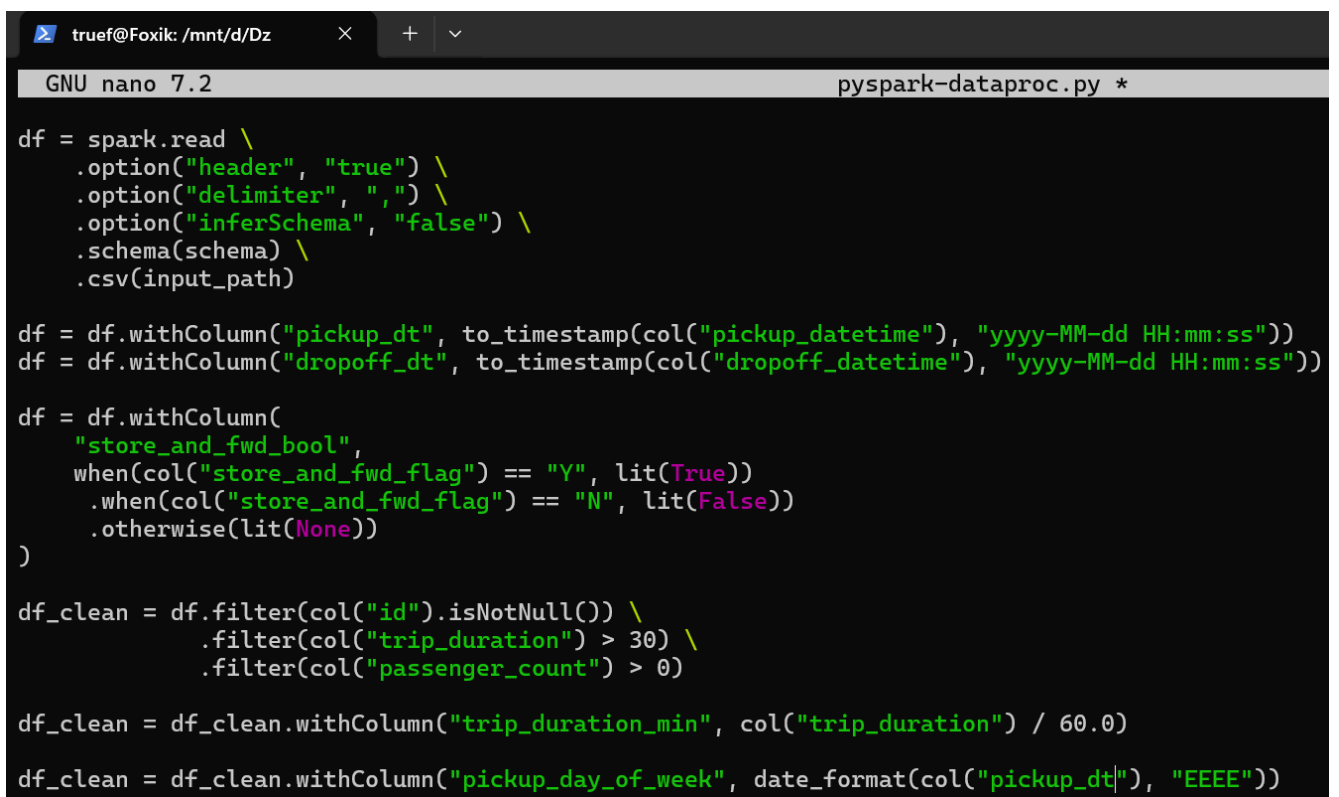
spark = SparkSession.builder \
    .appName("NYC_Taxi_Proc") \
    .enableHiveSupport() \
    .getOrCreate()

schema = StructType([
    StructField('id', StringType(), True),
    StructField('vendor_id', IntegerType(), True),
    StructField('pickup_datetime', StringType(), True),
    StructField('dropoff_datetime', StringType(), True),
    StructField('passenger_count', IntegerType(), True),
    StructField('pickup_longitude', StringType(), True),
    StructField('pickup_latitude', StringType(), True),
    StructField('dropoff_longitude', StringType(), True),
    StructField('dropoff_latitude', StringType(), True),
    StructField('store_and_fwd_flag', StringType(), True),
    StructField('trip_duration', IntegerType(), True)])

input_path = "s3a://for-dataproc-tat/in/NYC.csv"
output_path = "s3a://for-dataproc-tat/out/taxi"
```

Далее - чтение файла из `input_path` и необходимая обработка. Я бы предложила базовые действия (т.к. иное не указано в задании) - преобразование типов (`datetime` для двух столбцов - с временем и датой начала и окончания поездки).

Также можно выполнить преобразование типа для столбца с `store_and_fwd_flag`. В нем представлены два значения: Y (yes) / N (no). Логический тип данных в данном случае оптимальнее по занимаемому месту и возможностям для дальнейшей аналитики (фильтрация, например).



The screenshot shows a terminal window with a dark background. At the top, there's a window title bar with a blue icon, the text 'truef@Foxik: /mnt/d/Dz', and window control buttons. Below it is a header bar for the 'GNU nano 7.2' editor, with the filename 'pyspark-dataproc.py' on the right. The main area contains the following Python code:

```
df = spark.read \
    .option("header", "true") \
    .option("delimiter", ",") \
    .option("inferSchema", "false") \
    .schema(schema) \
    .csv(input_path)

df = df.withColumn("pickup_dt", to_timestamp(col("pickup_datetime"), "yyyy-MM-dd HH:mm:ss"))
df = df.withColumn("dropoff_dt", to_timestamp(col("dropoff_datetime"), "yyyy-MM-dd HH:mm:ss"))

df = df.withColumn(
    "store_and_fwd_bool",
    when(col("store_and_fwd_flag") == "Y", lit(True))
    .when(col("store_and_fwd_flag") == "N", lit(False))
    .otherwise(lit(None))
)

df_clean = df.filter(col("id").isNotNull()) \
    .filter(col("trip_duration") > 30) \
    .filter(col("passenger_count") > 0)

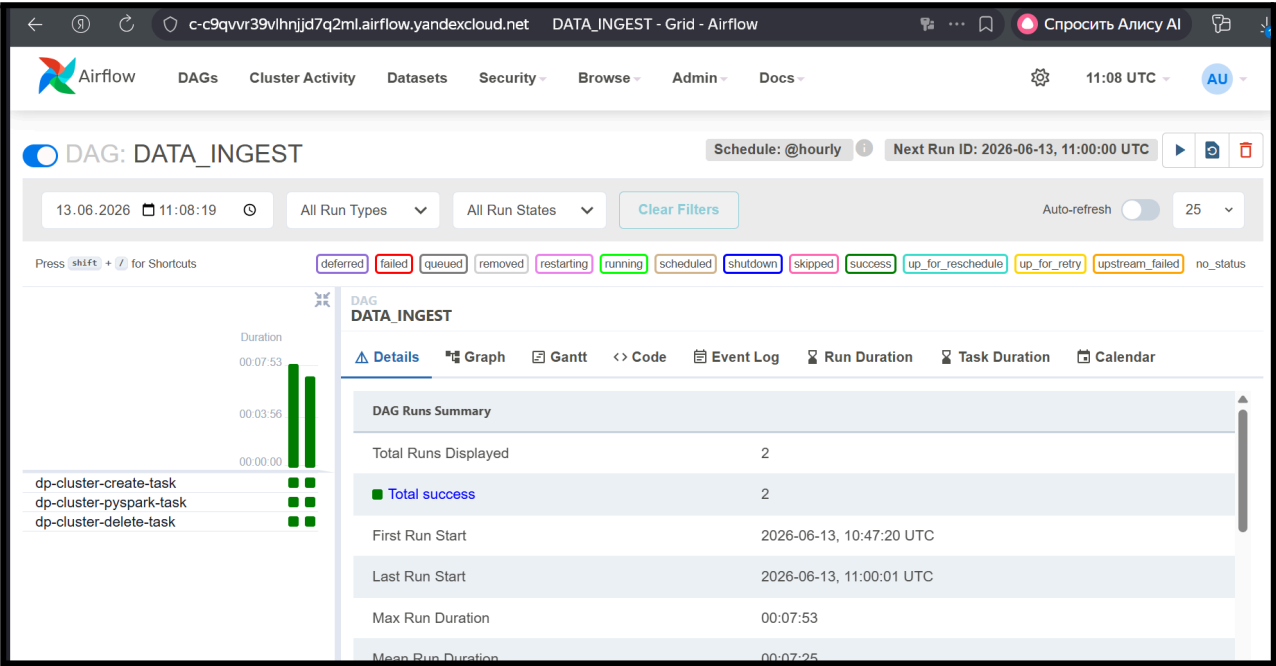
df_clean = df_clean.withColumn("trip_duration_min", col("trip_duration") / 60.0)

df_clean = df_clean.withColumn("pickup_day_of_week", date_format(col("pickup_dt"), "EEEE"))
```

Также я бы убрала строки, в которых длительность поездки менее 30 секунд (предположим, что наше исследование должно затрагивать только совершенные поездки, а менее 30 секунд - это явно ошибка или отказ). Еще можно убрать строки, в которых значение количества пассажиров нулевое.

Добавлю столбцы с длительностью поездки в минутах (удобнее, если потом для этих данных будем строить дашборды) и с наименованием дня недели (аналогично). Далее пишем обновленный файл в бакет и закрываем сессию.

Загрузила в бакет DAG и PySpark-задание. Выполнила DAG



Все работает корректно. Теперь проверяю, что в out в бакете. Вот мои данные:

The screenshot shows the Yandex Object Storage interface for the bucket 'for-dataproc-tat'. The interface includes a top navigation bar with links to Object Storage, ... / for-dataproc-tat / out / taxi, and buttons for 'Обработать в YQ', 'Создать папку', and 'Загрузить'. The main content area displays a list of objects in the 'out' bucket. The table below shows the details of the objects:

Имя	Размер	Класс хранилища	Последнее изменение
_SUCCESS	12.35 КБ	Стандартное	13.06.2026, в 14:04
part-00000-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.28 МБ	Стандартное	13.06.2026, в 14:04
part-00001-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.28 МБ	Стандартное	13.06.2026, в 14:04
part-00002-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.28 МБ	Стандартное	13.06.2026, в 14:04
part-00003-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.28 МБ	Стандартное	13.06.2026, в 14:04
part-00004-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.27 МБ	Стандартное	13.06.2026, в 14:04
part-00005-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	12.28 МБ	Стандартное	13.06.2026, в 14:04
part-00006-30753852-4d3b-450d-9a62-1f776aa860cc-c000.sn...	10.67 МБ	Стандартное	13.06.2026, в 14:04

Итог: данные из внешнего источника успешно обработаны Yandex Data Processing при помощи Apache AirFlow

Задание 3. Работа с топиками Apache Kafka с помощью PySpark-заданий в Yandex Data Processing

Для начала подготовительные шаги: выдала необходимые права сервисному аккаунту, настроила группы безопасности, создала сеть и подсеть, настроила NAT-шлюз и таблицу маршрутизации.

Следующий шаг - создание бакета Object Storage и двух кластеров, Yandex Data Processing Managed Service for Apache Kafka. Кластеры запущены и живы

The screenshot shows the Yandex Data Processing console interface. The left sidebar contains navigation links: Обзор, Подкластеры, Хосты, Операции, Мониторинг, Права доступа, Логи, Топология, Задания, Документация, and links to cluster setup and host classes. The main panel displays the overview for cluster 'dataproc726', which is 'Alive'. It includes fields for Identifier, Name, Creation Date, Environment, Service Account, and Bucket Name. A 'Доступность' (Availability) section shows 'Alive' and a message that all hosts are working. A 'Настройки' (Settings) section lists Version, Availability Zone, and Services. A pricing sidebar on the right shows a cost of 30,59 RUB per hour and a breakdown of resources: VMs (12,40 RUB), Hosts (1,38 RUB), and RAM (13,20 RUB). A notification about a price change for services starting May 1, 2026, is also present.

Поле	Значение
Идентификатор	c9qt8p5b6uld39t5ut5b
Имя	dataproc726
Дата создания	14.06.2026, в 09:28
Окружение	PRODUCTION
Сервисный аккаунт	dataproc-sa
Имя бакета	dataproc-bucket

Ресурс	Стоимость
Вычислительные ресурсы обычной VM, Intel Ice Lake, 100% vCPU	12,40 Р
Вычислительные ресурсы хостов, Intel Ice Lake, 100% vCPU	1,38 Р
Вычислительные ресурсы обычной VM, Intel Ice Lake, RAM	13,20 Р

The screenshot shows the Yandex Managed Service for Kafka console interface. The left sidebar contains navigation links: Обзор, Топики, Коннекторы, Хосты, Пользователи, Мониторинг, Права доступа, Логи, Операции, Обслуживание, and Трансферы. The main panel displays the overview for cluster 'kafka837', which is 'Alive'. It includes fields for Name, Identifier, Creation Date, Environment, Version, Schema Registry, Kafka Rest API, and Kafka UI. A 'Доступность' (Availability) section shows 'Alive' and a message that all hosts are working. An 'Обслуживание' (Maintenance) section shows the maintenance window as 'вторник 01:00 - 02:00'. A pricing sidebar on the right shows a cost of 5 657,47 RUB per month and a breakdown of resources: Hosts (2 505,60 RUB), RAM (2 649,60 RUB), and SSD storage. A notification about a price change for services starting May 1, 2026, is also present.

Поле	Значение
Имя	kafka837
Идентификатор	c9qa787244h1ra4dorfn
Дата создания	14.06.2026, в 09:29
Окружение	PRODUCTION
Версия	3.9.2
Реестр схем данных	Нет
Kafka Rest API	Нет
Kafka UI	Выключено

Ресурс	Стоимость
Managed Service for Apache Kafka®. Вычислительные ресурсы хостов-брокеров Apache Kafka®, AMD Zen 4, 100% vCPU	2 505,60 Р
Managed Service for Apache Kafka®. Вычислительные ресурсы хостов-брокеров Apache Kafka®, AMD Zen 4, RAM	2 649,60 Р
Managed Service for Apache Kafka®. Хранилище на сетевых SSD-дисках	

Теперь необходимо подготовить кластер Kafka к работе: создать пользователя и топик. После этого я готовлю PySpark задания.

1. kafka-write.py

Генерирую нужный мне объем JSON-данных

```
truef@Foxik: /mnt/d/Dz
GNU nano 7.2 kafka-write.py
#!/usr/bin/env python3

from pyspark.sql import SparkSession
from pyspark.sql.functions import to_json, col, struct
import json
import uuid

def generate_loan_data(count=60000):
    data = []
    regions = ["DE-HE", "FR-PAR", "US-NY", "GB-LON", "ES-MAD"]
    risks = ["low", "medium", "high"]
    for i in range(count):
        record = {
            "application_id": f"loan_{uuid.uuid4().hex[:8]}",
            "customer": {
                "customer_id": f"cust_{i % 1000}",
                "region": regions[i % len(regions)]
            },
            "loan": {
                "amount": 1000 + (i % 5000) * 100,
                "term_months": 12 + (i % 48)
            },
            "scoring": {
                "score": 500 + (i % 300),
                "risk_level": risks[i % len(risks)]
            },
            "documents": [
                {"type": "passport", "status": "verified" if i % 2 == 0 else "pending"},
                {"type": "income_proof", "status": "missing" if i % 5 == 0 else "verified"}
            ],
            "decision_status": "manual_review" if i % 10 == 0 else "approved",
            "submitted_at": "2026-05-01T10:15:11Z"
        }
        data.append(record)
    return data
```

И превращаю их в датафрейм, запустив spark-сессию. Отправляю в Kafka.

```

def main():
    spark = SparkSession.builder.appName("dataproc-kafka-write-app").getOrCreate()
    raw_data = generate_loan_data(60000)

    df = spark.createDataFrame(raw_data)

    df_kafka = df.select(to_json(struct([col(c).alias(c) for c in df.columns])).alias('value'))

    print(f"Generated {df.count()} records. Sending to Kafka...")

    df_kafka.write.format("kafka") \
        .option("kafka.bootstrap.servers", "rc1b-tbmlun786dms0qsg.mdb.yandexcloud.net:9091") \
        .option("topic", "dataproc-kafka-topic") \
        .option("kafka.security.protocol", "SASL_SSL") \
        .option("kafka.sasl.mechanism", "SCRAM-SHA-512") \
        .option("kafka.sasl.jaas.config",
                "org.apache.kafka.common.security.scram.ScramLoginModule required "
                "username=user1 "
                "password=password1 "
                ";") \
        .save()
    print("Successfully wrote data to Kafka!")

if __name__ == "__main__":
    main()

```

2. kafka-read-batch.py

Для чтения сообщений с самого начала

```

GNU nano 7.2 kafka-read-batch.py
#!/usr/bin/env python3

from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, ArrayType, MapType
from pyspark.sql.functions import from_json, col, explode

def main():
    spark = SparkSession.builder.appName("dataproc-kafka-read-batch-app").getOrCreate()

    schema = StructType([
        StructField("application_id", StringType()),
        StructField("customer", StructType([
            StructField("customer_id", StringType()),
            StructField("region", StringType())
        ])),
        StructField("loan", StructType([
            StructField("amount", IntegerType()),
            StructField("term_months", IntegerType())
        ])),
        StructField("scoring", StructType([
            StructField("score", IntegerType()),
            StructField("risk_level", StringType())
        ])),
        StructField("documents", ArrayType(StructType([
            StructField("type", StringType()),
            StructField("status", StringType())
        ]))),
        StructField("decision_status", StringType()),
        StructField("submitted_at", StringType())
    ])

    [ Read 69 lines ]

```

```

df_raw = spark.read.format("kafka") \
    .option("kafka.bootstrap.servers", "rc1b-tbmlun786dms0qsg.mdb.yandexcloud.net:9091") \
    .option("subscribe", "dataproc-kafka-topic") \
    .option("kafka.security.protocol", "SASL_SSL") \
    .option("kafka.sasl.mechanism", "SCRAM-SHA-512") \
    .option("kafka.sasl.jaas.config",
            "org.apache.kafka.common.security.scram.ScramLoginModule required "
            "username=user1 "
            "password=password1 "
            ";") \
    .option("startingOffsets", "earliest") \
    .load()

df_parsed = df_raw.selectExpr("CAST(key AS STRING)", "CAST(value AS STRING) as json_value") \
    .withColumn("data", from_json(col("json_value"), schema)) \
    .select("data.*")

df_flat = df_parsed.select(
    col("application_id"),
    col("customer.customer_id").alias("customer_id"),
    col("customer.region").alias("customer_region"),
    col("loan.amount").alias("loan_amount"),
    col("loan.term_months").alias("loan_term_months"),
    col("scoring.score").alias("score"),
    col("scoring.risk_level").alias("risk_level"),
    col("documents"),
    col("decision_status"),

```

```

df_flat.printSchema()
print(f"Total records read: {df_flat.count()}")
output_path = "s3a://dataproc-bucket/kafka-batch-output-flat"
df_flat.write.mode("overwrite").parquet(output_path)
print(f>Data saved to {output_path}")

if __name__ == "__main__":
    main()

```

Запускаю первое задание (kafka-write) - успешно

cloud-truefoxik

tati

Yandex Data Processing / Кластеры / dataproc726 / Задания / 2026-06-14T07:11:47.238Z

Перезапустить задание

dataproc726

Alive

Обзор

Подкластеры

Хосты

Операции

Мониторинг

Права доступа

Логи

Топология

Задания

Документация

Как устроен кластер в Yandex Data Processing

Классы хостов для кластеров

Обзор 2026-06-14T07:11:47.238Z

Идентификатор

c9q1fq1976eg4t89okq5

Имя

2026-06-14T07:11:47.238Z

Имя кластера

dataproc726

Статус

Done

Время выполнения

21.35 секунда

Начало

14.06.2026, в 10:12

Конец

14.06.2026, в 10:13

Пользователь

Татьяна Макаркина

Тип задания

PYSPARK

Аргументы задания

Настройки

spark.submit.deployMode: cluster

Main python файл

s3a://dataproc-bucket/kafka-write.py

Логи выполнения задания

Чтобы проверить, что данные в топик успешно записаны, создаю второе задание (kafka-read-batch). Оно также успешно выполняется

The screenshot shows the Yandex Data Processing console interface. On the left is a sidebar with navigation options: Обзор, Подкластеры, Хосты, Операции, Мониторинг, Права доступа, Логи, Топология, and Задания (selected). Below the sidebar is a 'Документация' section with a link 'Как устроен кластер в Yandex Data Processing'. The main panel displays the details of a task named 'kafka-read-batch' with ID '2026-06-14T07:22:40.183Z'. The task is in a 'Done' state. The 'Аргументы задания' section shows the configuration: 'spark.submit.deployMode: cluster' and 'Main python файл: s3a://dataproc-bucket/kafka-read-batch.py'. The 'Логи выполнения задания' section is visible at the bottom.

Идентификатор	Имя	Имя кластера	Статус	Время выполнения	Начало	Конец	Пользователь	Тип задания
c9qrsqgdec6vsha73i5	2026-06-14T07:22:40.183Z	dataproc726	Done	29.44 секунд	14.06.2026, в 10:23	14.06.2026, в 10:23	Татьяна Макаркина	PYSPARK

Аргументы задания

Настройки	Main python файл
spark.submit.deployMode: cluster	s3a://dataproc-bucket/kafka-read-batch.py

Логи выполнения задания

Так как в моем случае объем данных изначально известен (ограничила его условиями генерации), стриминг не нужен. Поэтому читаю топики в пакетном режиме

Проверю, как все разложилось (скачала файл из бакета):

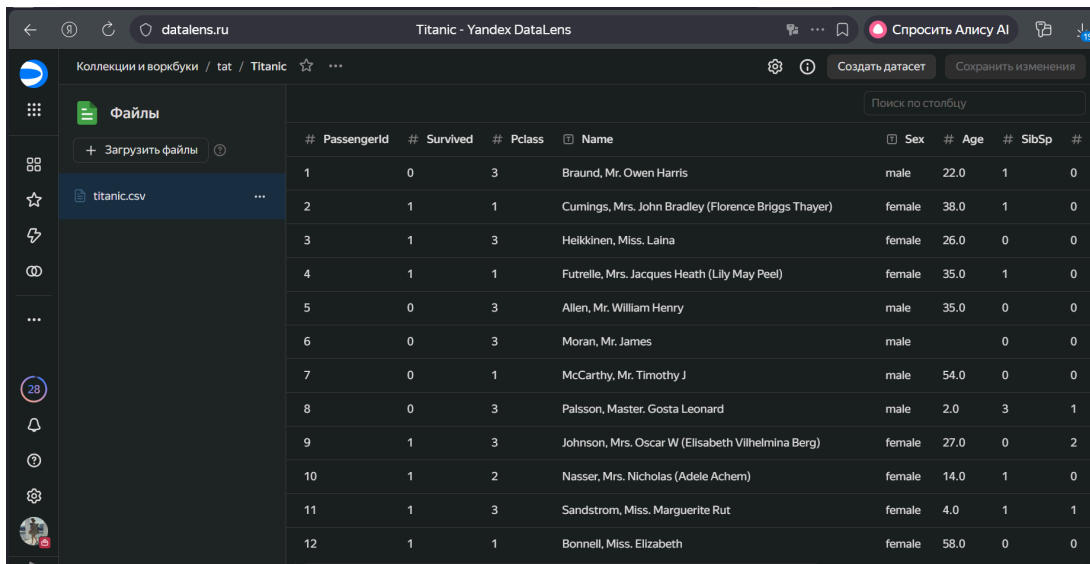
The screenshot shows a file explorer window with the file 'part-00000-3d0314f2-f0ce-472b-aeb5-d73a1c71de1a-c000.snappy.parquet' selected. Below the file name, there is a preview of the data in a table format. The table has columns: application_id, customer_id, customer_region, loan_amount, loan_term_months, score, risk_level, documents, decision_status, and submitted_at. The data is organized into 11 rows, each representing a loan record.

	application_id	customer_id	customer_region	loan_amount	loan_term_months	score	risk_level	documents	decision_status	submitted_at
1	loan_80c90412	cust_0	DE-HE	1000	12	500		[object Object],[object Object]	manual_review	2026-05-01T10
2	loan_f135b622	cust_696	FR-PAR	470600	44	796		[object Object],[object Object]	approved	2026-05-01T10
3	loan_2c93e8d8	cust_1	FR-PAR	1100	13	501		[object Object],[object Object]	approved	2026-05-01T10
4	loan_7b7eddce	cust_697	US-NY	470700	45	797		[object Object],[object Object]	approved	2026-05-01T10
5	loan_27b12d8a	cust_2	US-NY	1200	14	502		[object Object],[object Object]	approved	2026-05-01T10
6	loan_1b1da241	cust_698	GB-LON	470800	46	798		[object Object],[object Object]	approved	2026-05-01T10
7	loan_cd9005d1	cust_3	GB-LON	1300	15	503		[object Object],[object Object]	approved	2026-05-01T10
8	loan_01b520c7	cust_699	ES-MAD	470900	47	799		[object Object],[object Object]	approved	2026-05-01T10
9	loan_9a8305ab	cust_4	ES-MAD	1400	16	504		[object Object],[object Object]	approved	2026-05-01T10
10	loan_095c01f6	cust_700	DE-HE	471000	48	500		[object Object],[object Object]	manual_review	2026-05-01T10
11	loan_9909cree	cust_701	FR-PAR	471100	49	501		[object Object],[object Object]	approved	2026-05-01T10

Итог: данные успешно прочитаны

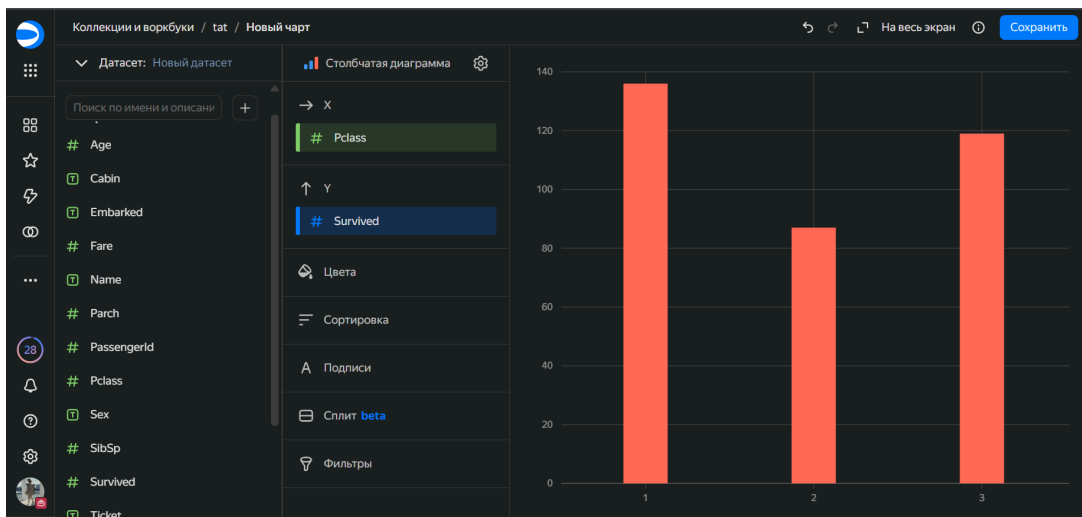
Задание 4. Визуализация в DataLens

В DataLens я буду работать с файлом titanic.csv. Для этого я создала воркбук, затем подключение и загрузила нужный мне файл

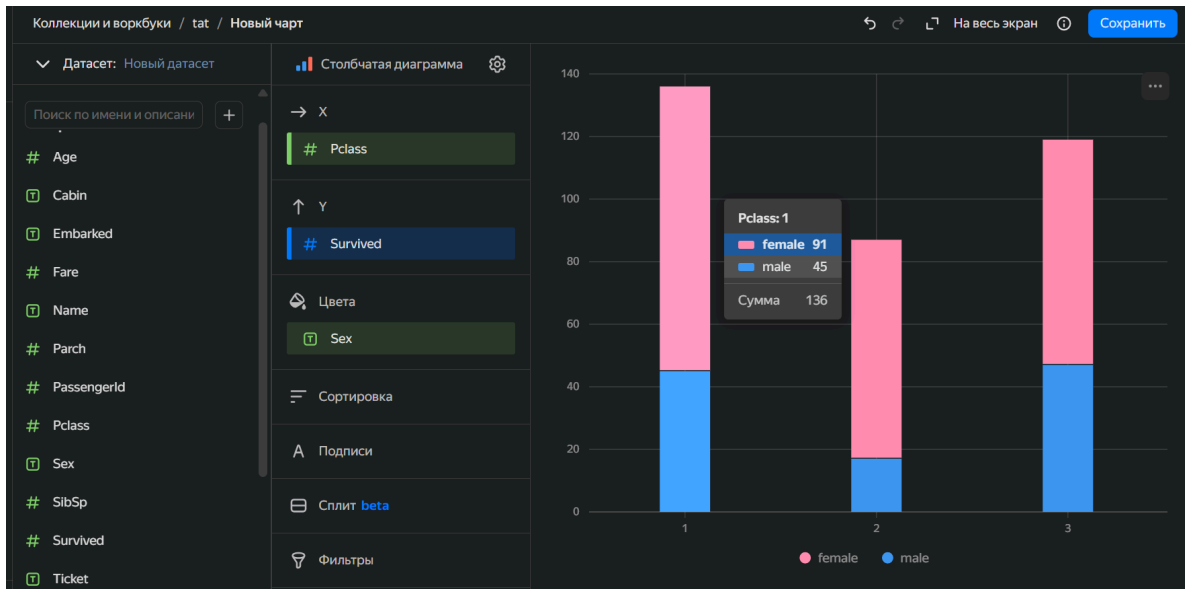


#	PassengerId	#	Survived	#	Pclass	Name	Sex	#	Age	#	SibSp	#
1		0		3		Braund, Mr. Owen Harris	male		22.0	1		0
2		1		1		Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female		38.0	1		0
3		1		3		Heikkinen, Miss. Laina	female		26.0	0		0
4		1		1		Futrelle, Mrs. Jacques Heath (Lily May Peel)	female		35.0	1		0
5		0		3		Allen, Mr. William Henry	male		35.0	0		0
6		0		3		Moran, Mr. James	male			0		0
7		0		1		McCarthy, Mr. Timothy J	male		54.0	0		0
8		0		3		Palsson, Master. Gosta Leonard	male		2.0	3		1
9		1		3		Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female		27.0	0		2
10		1		2		Nasser, Mrs. Nicholas (Adele Achem)	female		14.0	1		0
11		1		3		Sandstrom, Miss. Marguerite Rut	female		4.0	1		1
12		1		1		Bonnell, Miss. Elizabeth	female		58.0	0		0

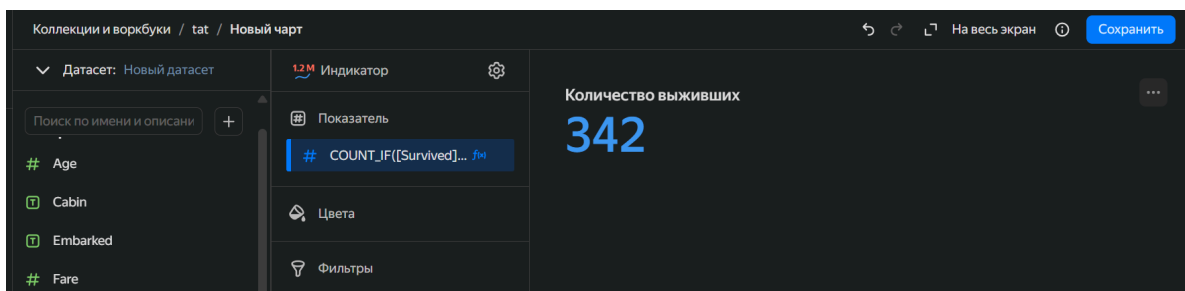
Далее нужно создать датасет на базе этого подключения. И перехожу в режим создания чартов. Самый простой чарт в данном случае - посмотреть, как меняется доля выживших пассажиров в зависимости от класса билета



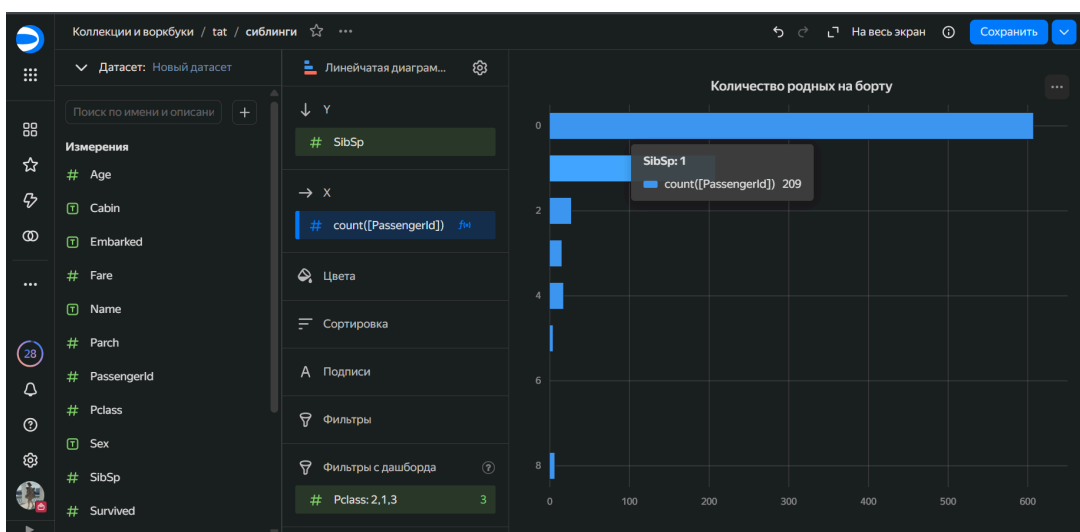
Можно добавить еще дополнительную цветовую сегментацию: розовым цветом будет показано количество пассажиров данного класса женского пола, голубым цветом - мужского



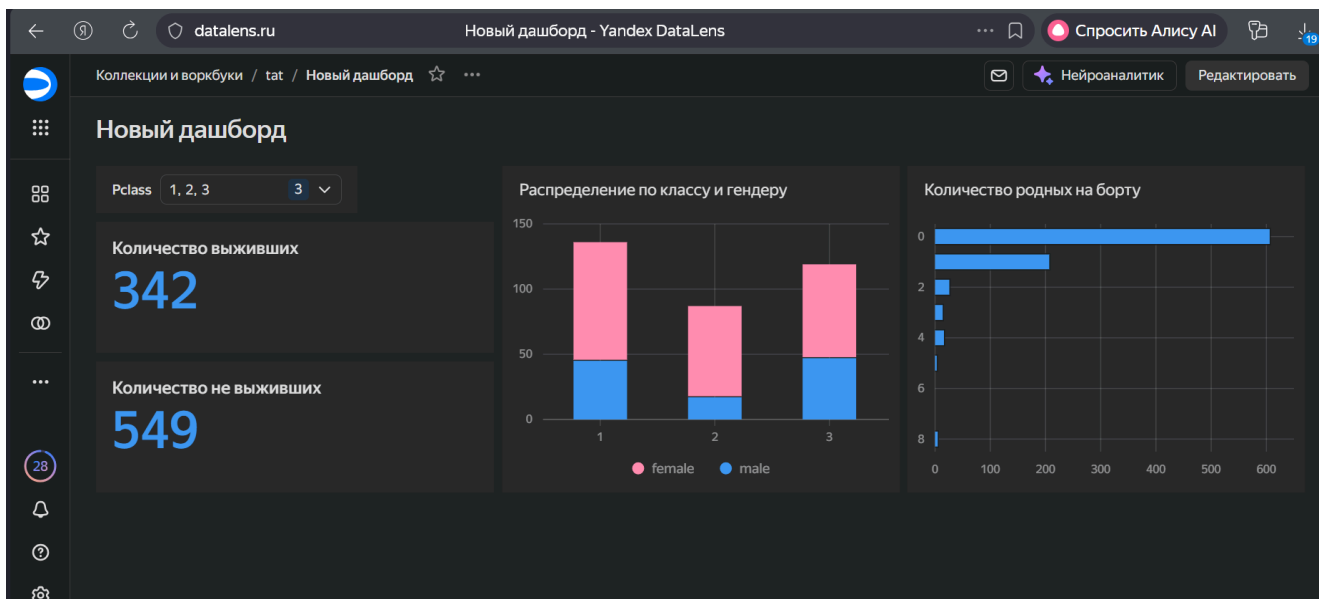
Сделала еще два простых поля-индикатора с количеством выживших и не выживших



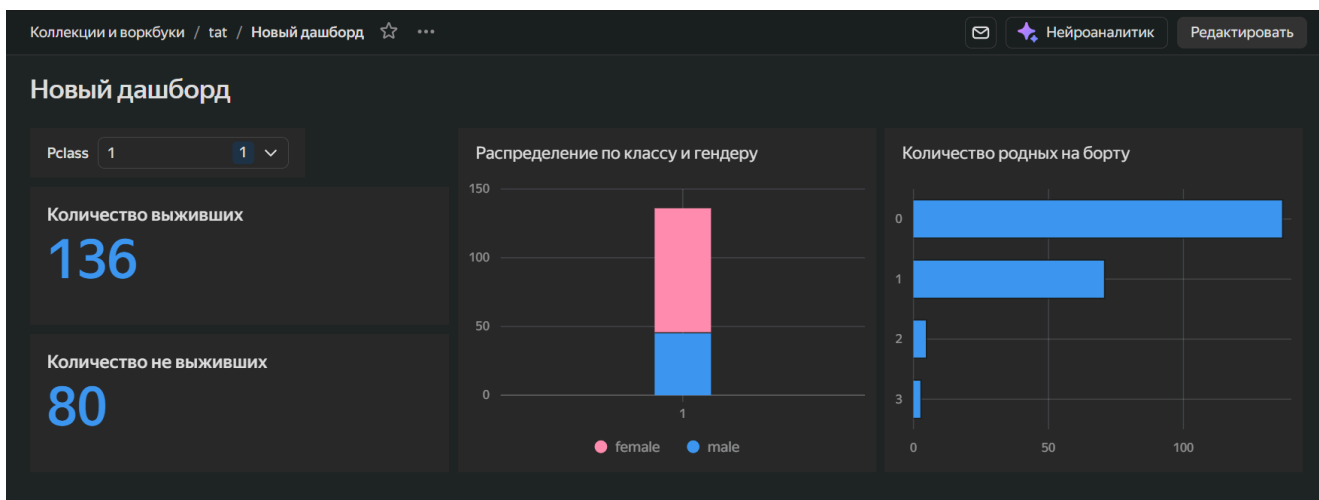
Добавила еще чарт с количеством родных на борту



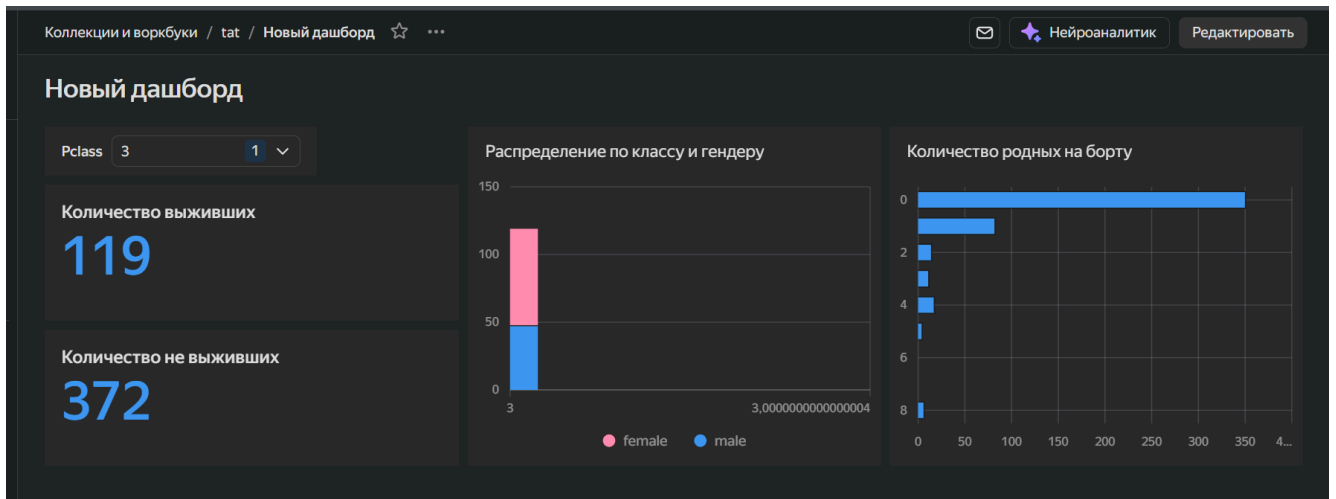
Пробую собрать небольшой дашборд:



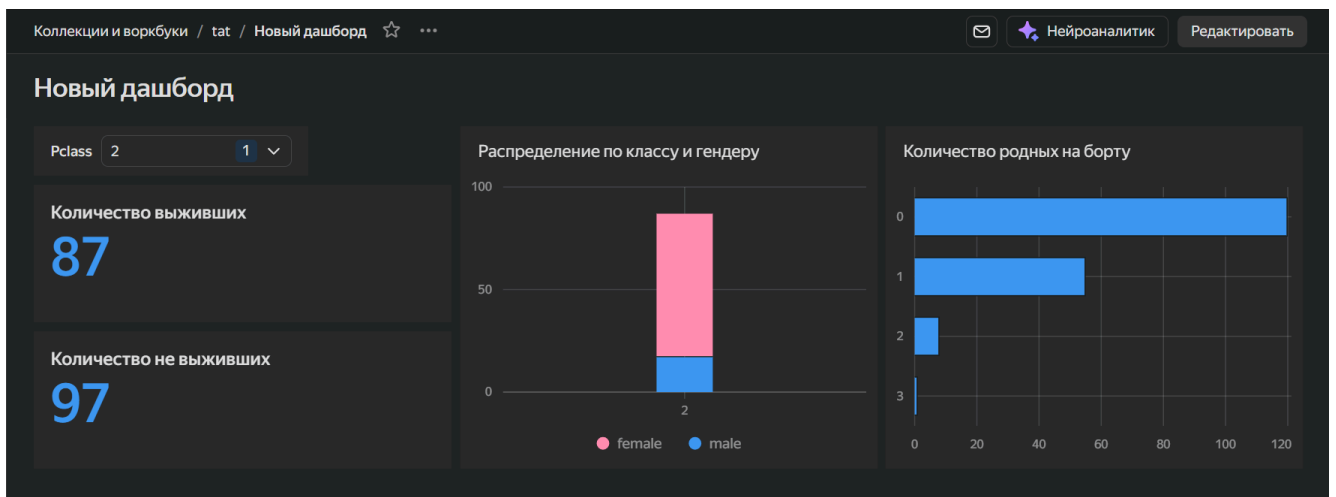
Отфильтруюсь по первому классу (все показатели изменились). Видно, что большинство выживших из этого класса - те, кто путешествовал в одиночку. Количество женщин среди выживших первого класса больше. И, наконец, выживших в этой категории больше, чем тех, кто не смог спастись



Посмотрю, как обстоят дела у третьего класса. Среди этих пассажиров гораздо больше тех, кто не смог спастись. Женщин также больше, как больше среди спасшихся и тех, кто путешествовал самостоятельно.



И для второго класса:



Ссылка на дашборд: <https://datalens.yandex/6pk4zv8lwzk0q>